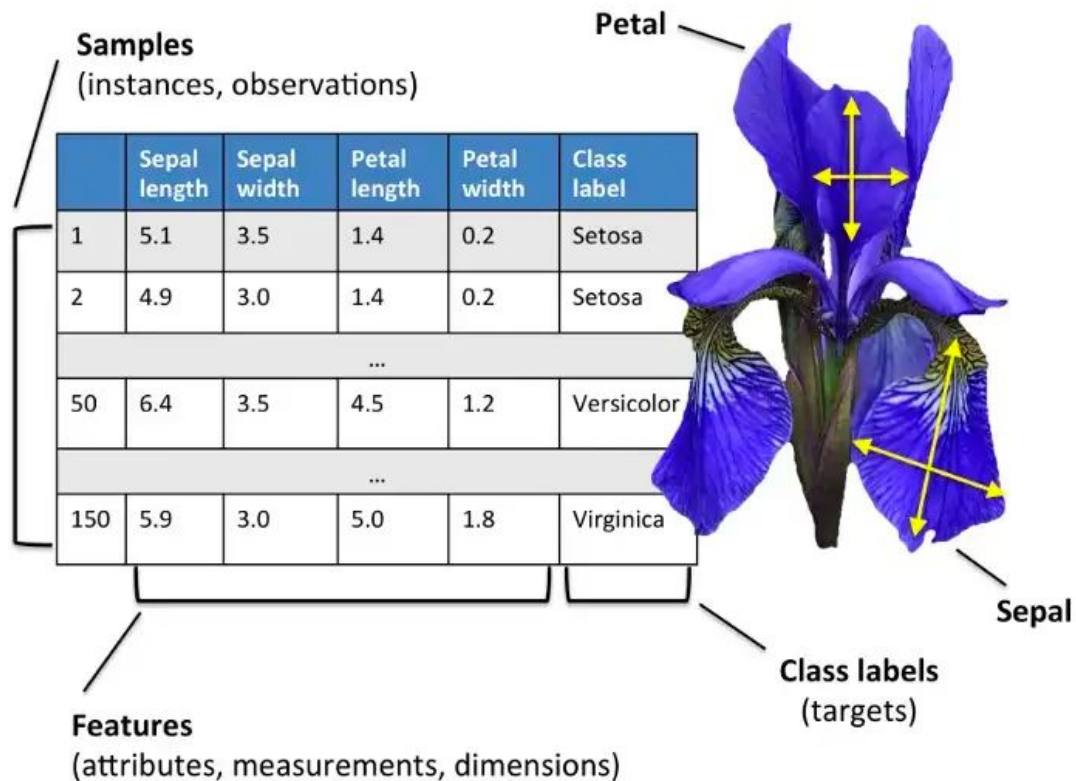


El “camino” de ML

Tecnología Digital VI

Licenciatura en Tecnología Digital - UTDT

Iris dataset



Out of the box

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = KNeighborsClassifier(n_neighbors=50)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
acc = accuracy_score(y_test, predictions)

print(f"Precisión del modelo: {acc:.1%}")
```

Precisión del modelo: 96.7%

y_test

```
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1, 2, 0, 0, 0, 0, 1, 2, 1, 1, 2, 0, 2,
       0, 2, 2, 2, 2, 2, 0, 0])
```

Camino de ML

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.



Definición del problema

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Definición del problema

Cuando se estudian los métodos de Machine Learning los problemas “nos llegan”.

En la práctica, detectar y desarrollar problemas es todo un universo. Hay libros, puestos de trabajo y materias enteras en torno a ciertos problemas.

Definir correctamente o buscarse un problema a resolver puede ser tan o más difícil que la resolución del mismo.

Definición del problema

"Quiero saber qué clientes se van a ir, necesito una solución de Churn Prediction"

"¿Qué vas a hacer con esos clientes?"

"Les voy a dar un descuento para ver si se quedan"

"Entonces necesitás un modelo que te diga quién va a cambiar de opinión con un descuento"

"Tengo una línea de producción, quiero clasificar estas imágenes en Correcto o cada una de los errores posibles"

"Perfecto, necesito entonces imágenes de entrenamiento con todos los errores posibles".

"No tengo, los errores son muy poquitos en proporción y además pueden aparecer errores nuevos que no conozco"

"Entonces necesitás un modelo que detecte anomalías"

"Necesito una solución de Computer Vision que marque polígonos en donde mi cultivo está seco. Necesito una solución de object detection."

"¿Podés definir qué es seco y qué no?"

"No, de hecho entre los diferentes analistas no nos ponemos de acuerdo en varias imágenes"

"Entonces vamos a hacer un modelo de segmentación donde tu output sea la probabilidad de pertenecer a la clase Secón."

"Uh sí, eso me sirve mucho más."

Adquisición de datos

- Definición del problema.
- **Adquisición de datos.**
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Adquisición de datos

Al igual que en “Definición del Problema”, nosotros vamos a asumir que esto ya está, pero en la práctica hay un montón de preguntas a responder que derivan en otras áreas de estudio y herramientas especializadas.

- ¿De dónde salen los datos?
- ¿Qué cantidad de datos tengo? Si son muchos, ¿cómo los disponibilizo?
- ¿Qué datos tengo en el momento de entrenamiento y cuáles voy a tener en el momento de inferencia?
- ¿Son confiables los datos? ¿cómo me entero si mis datos cambian o presentan alguna anomalía?
- ¿Qué datos me faltan para poder resolver nuevos problemas?

Las respuestas pueden condicionar fuertemente los siguientes pasos del camino.

Adquisición de datos

En la vida real no todo se reduce a un csv limpio (o por lo menos alguien tiene que reducirlo).

Las preguntas propuestas abren ramas enteras de estudio y especializaciones:

- Data Engineering: El desarrollo de toda la infraestructura informática para asegurar la disponibilidad de los datos.
- Data Quality: Herramientas, técnicas y procesos en la búsqueda de que los datos sirvan para cumplir con el objetivo que se los quiere usar.

Preprocesamiento de los datos

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Procesamiento de los datos

En la mayoría de los datos de Machine Learning los datos se preprocesan.

A veces puede ser por necesidad: mi modelo no admite variables categóricas o datos faltantes.

A veces puede ser por conveniencia: si creo nuevos features que sumen información mejora mi predicción.

Dada la estructura común de los problemas tabulars, podemos separar el preprocesamiento en:

- Preprocesamiento de filas.
- Preprocesamiento de columnas.

Preprocesamiento de filas

Preprocesar las filas significa realizar algún tipo de preprocesamiento sobre las muestras de nuestro dataset. Existen varios tipos de preprocesamiento de filas, algunos de ellos son:

- Imputación de valores. (puede verse también como preprocesamiento de columnas)
- Remoción de errores o outliers.
- Oversampling y Undersampling.
- Data Augmentation.

Datos faltantes

En muchos casos nos va a suceder que nuestro dataset tiene *datos faltantes*. La mayoría de los algoritmos no se lleva bien con un “no-dato”. En general, debemos resolver el dato faltante de alguna manera.

Algunas posibles estrategias:

- Utilizar algoritmos que permitan “NaNs”.
- Eliminar datos faltantes.
- Imputación de valores:
 - Imputaciones simples por media, mediana, moda.
 - Imputaciones más complejas por regresiones u otros modelos.

Datos faltantes

Utilizar algoritmos que permitan “NaNs”: Si bien hay algoritmos que manejan datos faltantes, por dentro están haciendo algún tipo de imputación o asunción que debemos entender.

Eliminar valores siempre es una opción pero abre diferentes preguntas:

- ¿Elimino la fila o elimino la columna?
- ¿Me pierdo todas esas filas de entrenamiento?
- Cuando esté en producción, ¿qué hago? ¿no te proceso porque te falta un dato?

Imputación de valores

Imputar datos significa rellenarlos de alguna manera. Siempre rellenar datos faltantes introduce algún tipo de sesgo con el que hay que lidiar.

Técnicas generales:

- Imputación por mediana.
- Imputación por media.
- Imputación por moda.
- Imputación por nueva categoría (solo en categóricas).

Luego existen técnicas más sofisticadas o dedicadas a ciertos tipos de datos (por ejemplo, series de tiempo).

Imputación de valores

Si trabajamos con DataFrames de Pandas es simple hacer varias de las imputaciones mencionadas.

Adicionalmente, la librería Scikit-Learn tiene un módulo impute con varias funcionalidades implementadas.

(<https://scikit-learn.org/stable/modules/impute.html>)

```
from sklearn.datasets import load_iris
from sklearn.impute import SimpleImputer
import numpy as np

iris = load_iris()
X = iris.data

np.random.seed(0)
mask = np.random.choice([True, False], size=X.shape, p=[0.9, 0.1])
X_with_missing = np.where(mask, X, np.nan)

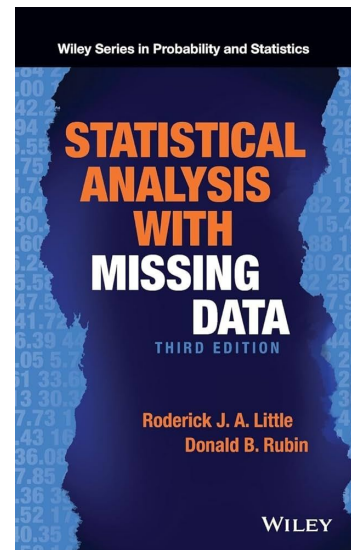
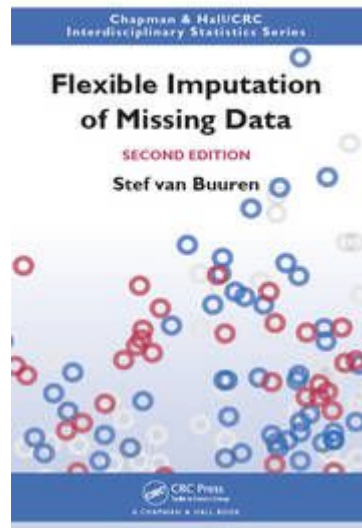
imputer = SimpleImputer(strategy='mean')

X_imputed = imputer.fit_transform(X_with_missing)
```

Imputación de valores

Como mucho de los temas que veremos, se puede ir mucho más profundo en técnicas más avanzadas y específicas para ciertas situaciones.

Incluso se puede ahondar en entender por qué y cómo (MCAR, MAR, MNAR) faltan los datos en nuestro dataset para elegir la técnica correcta.



Remoción de outliers

Un outlier es un valor que cae muy alejado del centro de una distribución.

En algunos casos la presencia de outliers puede ser un problema dado que hay algoritmos que son muy sensibles a la presencia de valores anómalos.

También es posible que un valor muy alejado de la distribución esté indicando otro tipo de problemas, como podría ser un error en la carga de datos.

Existen varios métodos de detección de outliers pero en general todos comparten la misma base estadística en donde se define un threshold de alejamiento de un atributo de la distribución como podría ser la media.

Remoción de outliers

En Scikit-Learn hay diferentes opciones para detectar outliers.

Una de ellas es la aplicación del método `robust_scale` del módulo `preprocessing`.

(https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.robust_scale.html)

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import robust_scale

iris = load_iris()
X = iris.data

X_scaled = robust_scale(X)

threshold = 3

X_filtered = X[abs(X_scaled) < threshold]
```

Oversampling y Undersampling

Algunos datasets resultan ser muy *desbalanceados*. Esto en general significa que en un problema de clasificación la distribución de las clases no es homogénea.

En estos casos se pueden utilizar técnicas de oversampling para obtener más muestras de las clases minoritarias o undersampling para remover muestras de las clases mayoritarias.

Algunas técnicas utilizadas para este fin:

- Oversampling: Random, SMOTE, ADASYN.
- Undersampling: Random, NearMiss.

Oversampling

La librería Imbalanced-learn (imblearn) está implementada sobre Scikit-learn y está pensada para tratar con dataset desbalanceados.

En https://imbalanced-learn.org/stable/over_sampling.html se pueden ver varios ejemplos y explicaciones de Oversampling.

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=5000, n_features=2, n_informative=2,
                           n_redundant=0, n_repeated=0, n_classes=3,
                           n_clusters_per_class=1,
                           weights=[0.01, 0.05, 0.94],
                           class_sep=0.8, random_state=0)

from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(random_state=0)
X_resampled, y_resampled = ros.fit_resample(X, y)
from collections import Counter
print(sorted(Counter(y).items()))
print(sorted(Counter(y_resampled).items()))

[(np.int64(0), 64), (np.int64(1), 262), (np.int64(2), 4674)]
[(np.int64(0), 4674), (np.int64(1), 4674), (np.int64(2), 4674)]
```

Undersampling

Análogamente, en https://imbalanced-learn.org/stable/under_sampling.html se pueden encontrar diversos ejemplos y explicaciones sobre las funcionalidades de Undersampling.

```
from collections import Counter
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=5000, n_features=2, n_informative=2,
                          n_redundant=0, n_repeated=0, n_classes=3,
                          n_clusters_per_class=1,
                          weights=[0.01, 0.05, 0.94],
                          class_sep=0.8, random_state=0)

from imblearn.under_sampling import ClusterCentroids
cc = ClusterCentroids(random_state=0)
X_resampled, y_resampled = cc.fit_resample(X, y)
print(sorted(Counter(y).items()))
print(sorted(Counter(y_resampled).items()))

[(np.int64(0), 64), (np.int64(1), 262), (np.int64(2), 4674)]
[(np.int64(0), 64), (np.int64(1), 64), (np.int64(2), 64)]
```


Data Augmentation

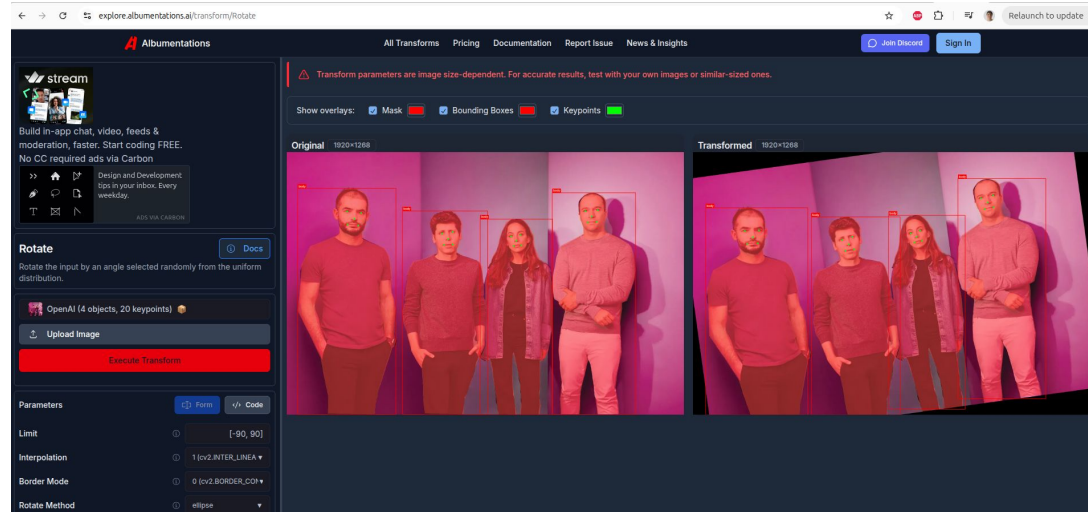
Otro preprocesamiento posible en las muestras es el de Data Augmentation. El mismo consiste en crear más muestras a partir de las originales para tener un entrenamiento más robusto.

Esta técnica es particularmente usada en los problemas de imágenes en dónde hay transformaciones muy simples de hacer sabiendo que las muestras generadas van a ser útiles cómo rotaciones, reflexiones, traslaciones.

Data Augmentation

Hay librerías como *fastai* que hacen estas transformaciones cuando tratan con un problema de imágenes.

Existen otras librerías como *Albumentations* que están dedicadas completamente al proceso de Data Augmentation.



Preprocesamiento de columnas

Es lo que en general se conoce como *Feature Engineering**. Es el preprocesamiento hecho para agregar, quitar o modificar columnas del dataset.

Algunos ejemplos:

- Feature Selection.
- Feature Extraction (en algunos lugares sinónimo de Feature Engineering).
- Feature Transformation.

Feature Selection

Es el proceso de seleccionar columnas para descartar algunas porque proveen poca información predictiva, aunque también existen otros motivos para su remoción (irrelevancia, correlación, performance, etc).

En general en la literatura se suelen separar los métodos de Feature Selection en 3 técnicas (y en realidad solo 1 es de preprocesamiento):

- Filter Methods: preprocesamiento.
- Embedded Methods: sucede durante la optimización del modelo.
- Wrapper Methods: *postprocesamiento.

Feature Selection – Filter Methods

Los métodos de filtrado son los únicos que son de preprocesamiento propiamente dicho. La idea es descartar columnas en base a algún criterio estadístico previo al entrenamiento del modelo.

Por ejemplo, una columna con poca o nula varianza no suele ser de gran utilidad (¿Por qué?). Se pueden filtrar directamente las columnas que su varianza sea menor a un cierto límite.

También se pueden *tirar* columnas que tengan poca correlación con la variable dependiente (¿Por qué?), aunque hay que tener cuidado de no estar tirando información importante.

Feature Selection – Filter Methods

En https://scikit-learn.org/stable/modules/feature_selection.html se pueden ver algunos métodos de Feature Selection. Las primeras funcionalidades presentadas son del tipo Filter Methods.

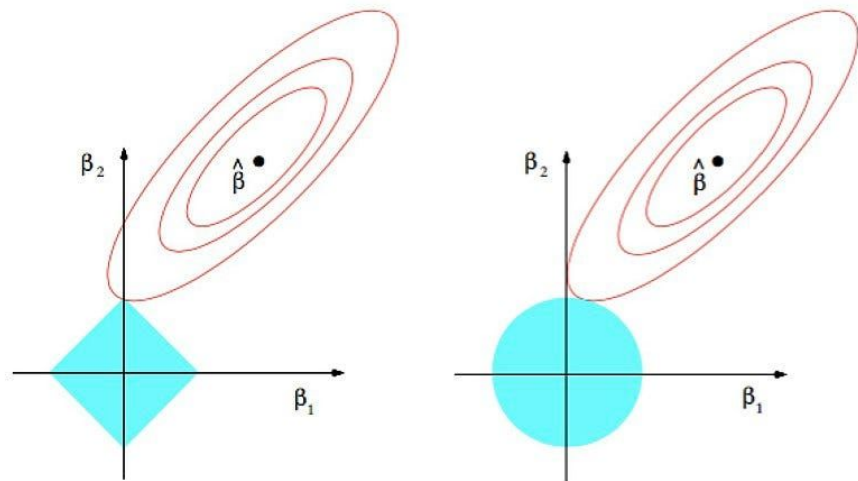
```
from sklearn.feature_selection import VarianceThreshold
X = [[0, 0, 1],
      [0, 1, 0],
      [1, 0, 0],
      [0, 1, 1],
      [0, 1, 0],
      [0, 1, 1]]
sel = VarianceThreshold(threshold=(.8 * (1 - .8)))
sel.fit_transform(X)

array([[0, 1],
       [1, 0],
       [0, 0],
       [1, 1],
       [1, 0],
       [1, 1]])
```

Feature Selection – Embedded Methods

Los Embedded Methods suceden durante el entrenamiento del modelo. Son métodos que agregan ciertas particularidades al entrenamiento del modelo para que se descarten columnas de no tanta importancia.

Veremos más detalles cuando veamos los modelos.



Feature Selection – Wrapper Methods

Luego de entrenar un modelo, se lo puede utilizar para ir descartando features viendo, por ejemplo, qué tanto empeora el predictor al extraer ciertas columnas.

Existen diversos algoritmos y técnicas para eliminar columnas con esta estrategia:

- Best Subset Selection
- Forward Selection
- Backward Selection

Best Subset Selection

Una opción es probar todas las combinaciones de features

Algorithm 6.1 *Best subset selection*

1. Let $\mathcal{M}_0$ denote the *null model*, which contains no predictors. This model simply predicts the sample mean for each observation.
 2. For $k = 1, 2, \dots, p$:
 - (a) Fit all $\binom{p}{k}$ models that contain exactly k predictors.
 - (b) Pick the best among these $\binom{p}{k}$ models, and call it $\mathcal{M}_k$. Here *best* is defined as having the smallest RSS, or equivalently largest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

Pero en general es prohibitivamente costoso

Forward Selection

Otra opción es ir agregando variables de a una al modelo

Algorithm 6.2 *Forward stepwise selection*

1. Let $\mathcal{M}_0$ denote the *null* model, which contains no predictors.
 2. For $k = 0, \dots, p - 1$:
 - (a) Consider all $p - k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
 - (b) Choose the *best* among these $p - k$ models, and call it $\mathcal{M}_{k+1}$. Here *best* is defined as having smallest RSS or highest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

Menos costoso computacionalmente, pero es una estrategia golosa que podría no encontrar el óptimo.

Backward Selection

Alternativamente, podemos comenzar con todas las variables e ir quitando

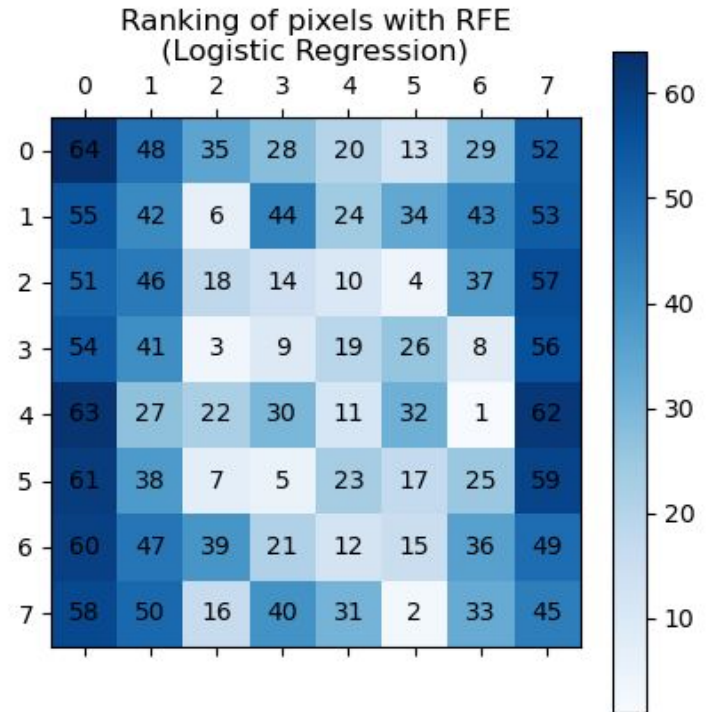
Algorithm 6.3 *Backward stepwise selection*

1. Let $\mathcal{M}_p$ denote the *full* model, which contains all p predictors.
 2. For $k = p, p - 1, \dots, 1$:
 - (a) Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$, for a total of $k - 1$ predictors.
 - (b) Choose the *best* among these k models, and call it $\mathcal{M}_{k-1}$. Here *best* is defined as having smallest RSS or highest R^2 .
 3. Select a single best model from among $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2 .
-

Feature Selection

Scikit-Learn tiene funcionalidades implementadas para ayudar en este tipo de Feature Selection.

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.RFE.html



Feature Extraction

Es el proceso de extraer features nuevas a partir del dataset original. El objetivo es obtener mejor poder de predicción agregando información que no podría ser capturada por el modelo elegido.

Hay muchas maneras diferentes de realizar este proceso, algunos ejemplos:

- Crear nuevos features como combinación de los originales porque el modelo elegido no puede combinarlos. Ejemplo: elevar variables al cuadrado en modelos lineales.
- Crear nuevos features de información oculta en un feature original. Ejemplo: a partir de una fecha saber si es feriado o no.
- Crear nuevos features buscando maximizar la información y reduciendo la correlación (lo vemos en un par de clases).
- Agregar nuevas fuentes de datos externas.

Feature Extraction

https://scikit-learn.org/stable/modules/feature_extraction.html tiene funcionalidades de Scikit-learn para esto.

https://docs.fast.ai/tabular.core.html#add_datepart es un ejemplo que trabaja sobre fechas de manera automática.

add_datepart

```
def add_datepart(
    df, field_name, prefix=None, drop:bool=True, time:bool=False
):
```

Helper function that adds columns relevant to a date in the column `field_name` of `df`.

For example if we have a series of dates we can then generate features such as `Year`, `Month`, `Day`, `Dayofweek`, `Is_month_start`, etc as shown below:

```
df = pd.DataFrame({'date': ['2019-12-04', None, '2019-11-15', '2019-10-24']})
df = add_datepart(df, 'date')
df.head()
```

	Year	Month	Week	Day	Dayofweek	Dayofyear	Is_month_end	Is_month_start	Is_quarter_end	Is_quarter_start
0	2019.0	12.0	49.0	4.0	2.0	338.0	False	False	False	False
1	NaN	NaN	NaN	NaN	NaN	NaN	False	False	False	False
2	2019.0	11.0	46.0	15.0	4.0	319.0	False	False	False	False
3	2019.0	10.0	43.0	24.0	3.0	297.0	False	False	False	False

Feature Transformation

En Feature Transformation no se crean ni se descartan columnas si no que se transforman de alguna manera los features originales.

Hay muchos tipos de transformaciones dependiendo el contexto de aplicación, la naturaleza de los datos y el modelo de machine learning elegido:

- Scaling o Normalization
- Log transformation
- Binarization
- Quantization
- One Hot Encoding

Feature Scaling

Varios modelos utilizan los valores absolutos de los features para realizar operaciones (medidas de distancia, gradiente descendiente). Si los features están en órdenes de magnitud distinta, el modelo le dará una importancia ficticia a un feature por sobre otro.

Feature Scaling es tomar una columna y transformar todas sus filas para escalar los valores.

- Min-Max Scaling: Escala todos los valores a un cierto rango preservando las distancias.
- Standard Scaling: centra los datos para que tengan media 0 y desviación estandar 1.

Feature Scaling

Más allá de que sean fórmulas simples, varias librerías hacen el trabajo de Feature Scaling por nosotros.

```
from sklearn.preprocessing import MinMaxScaler

X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])

min_max_scaler = MinMaxScaler()
X_train_minmax = min_max_scaler.fit_transform(X_train)
X_train_minmax
```

```
array([[0.5       ,  0.       ,  1.       ],
       [1.       ,  0.5     ,  0.33333333],
       [0.       ,  1.       ,  0.       ]])
```

Log Transformation

A veces las distribuciones de los datos tienen una distribución muy asimétrica con largas colas positivas. Ese tipo de valores atípicos puede ser determinante para ciertos modelos que le den demasiada importancia.

Una transformación logarítmica puede ayudar a modificar la distribución para equilibrar la importancia de las muestras (o también para trabajar con una métrica más simple).

Quantization

En muchas situaciones no importa el valor numérico real de una columna sino que es más útil saber *a qué grupo* pertenece, o incluso simplemente saber si supera cierto threshold o no.

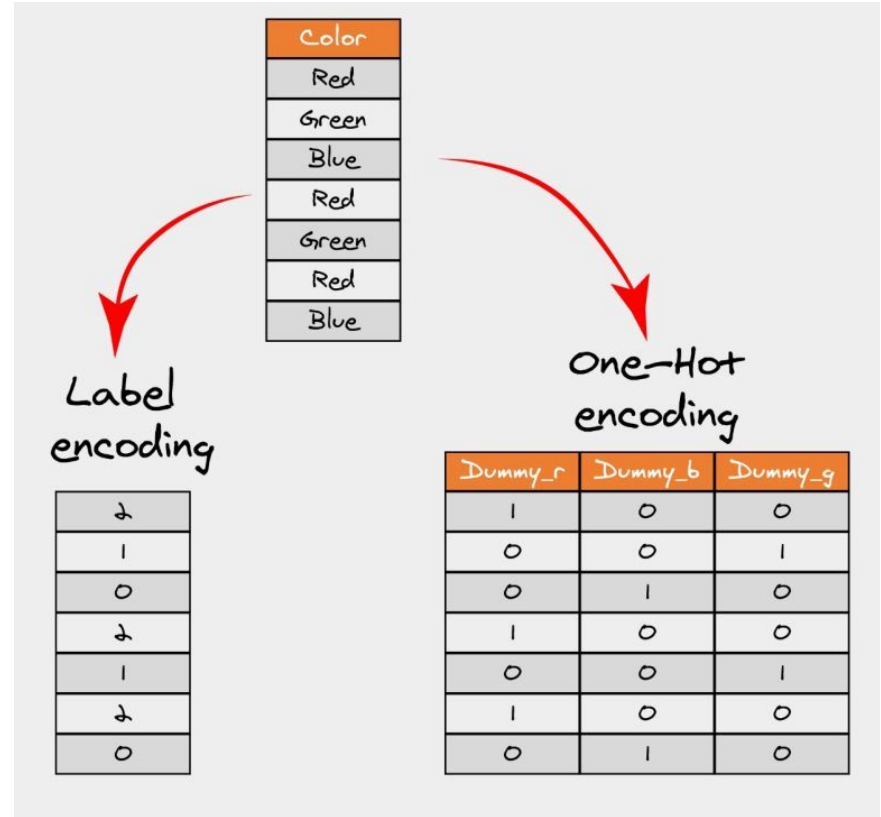
- Quantization: Se agrupan las observaciones en *bins* (Jovenes, Adultos, Ancianos).
- Binarization: Se crea una nueva variable que indica si la original es mayor o no a cierto valor.

One Hot Encoding

Muchos modelos de Machine Learning no están pensando para trabajar con variables no numéricas y, por ende, no pueden manejar variables categóricas. En estos casos, transformaremos las variables categóricas a numéricas con alguna estrategia.

Ejemplos:

- One Hot Encoding
- Label Encoding



Data Leakage

El momento en el que manipulamos las variables de entrada es cuando más podemos estar expuestos a *Data Leakage*.

Data Leakage significa que nuestro modelo entrena con información que no debería tener disponible, no porque queramos ocultarsela sino porque realmente no va a estar disponible al momento de predecir en producción.

En general un Data Leakage proviene de utilizar información del conjunto de test para entrenar, aunque puede haber otras fuentes.

Data Leakage

Algunos ejemplos de Data Leakage:

- Hacer Feature Selection utilizando información de la variable proveniente del conjunto de test.
- Hacer Feature Scaling utilizando todo el dataset.
- Romper la temporalidad de los datos.
- Target Leakages más sutiles como procesos que llenan campos en cierto orden.

Modelos de ML

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Eligiendo modelo

En realidad, seleccionar la clase del modelo, su algoritmo de optimización y su métrica de optimización no son pasos separados, pero hacemos mención de cada uno para entender su importancia y lo que implica en la práctica.

- Selección de la clase de modelo: Elijo qué modelo/s de ML voy a utilizar.
- Algoritmo de optimización: ¿Cómo encuentro el mejor modelo dentro de la clase que elegí?
- Métrica de optimización del modelo: ¿Cómo mido qué es el mejor modelo?

Selección de la clase de modelo

En algunos casos no solo el problema define el modelo sino también los datos disponibles. Hay modelos más adecuados para cuando tenemos un cierto tipo de datos, hay modelos más adecuados cuando tenemos pocos o muchos datos, etc.

En los vaivenes de este camino de Machine Learning, la elección de una clase de modelo a veces puede tener efecto en el preprocesamiento de los datos. Hay modelos que, por su funcionamiento, *necesitan* que todos los features estén en la misma escala.

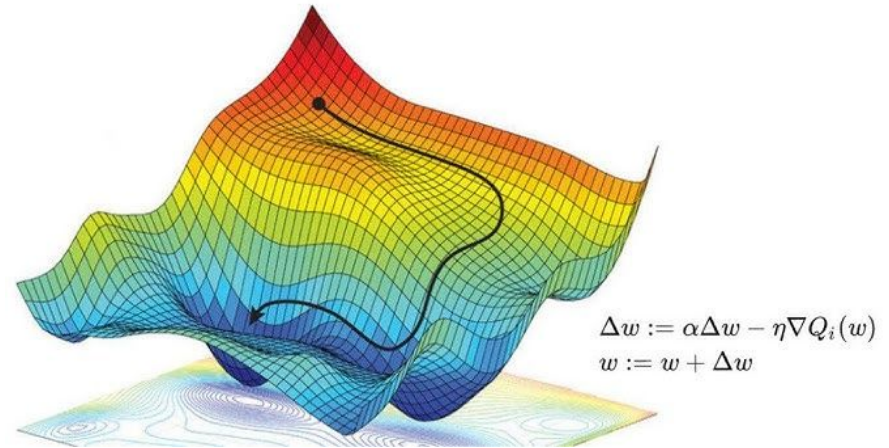
Hay diversos motivos para elegir una u otra clase de modelo, pero para eso tenemos que primero ver más modelos.

Algoritmo de optimización del modelo

Una vez elegido un modelo, de alguna manera tiene que *aprender*. Necesitamos un algoritmo para optimizar los parámetros del modelo.

Esto no es algo independiente de la clase de modelo. En general, cada clase ya viene aparejada con su algoritmo ideal.

Sin embargo, es importante conocer las limitaciones y necesidades del algoritmo subyacente porque eso puede impactar en otros pasos del camino como el preprocesamiento de los datos.



Métrica de optimización del modelo

De la mano con el algoritmo de optimización, está la métrica de optimización del modelo (que no necesariamente es la métrica con la que evaluamos el modelo).

Es lo que se conoce como *loss function* o *cost function*. Es la función que guía al algoritmo de optimización para conseguir los mejores parámetros.

Nuevamente, esto suele venir *regalado* cuando elegimos una clase de modelo, pero es importante entender que hay diferentes alternativas y cuál es la relación de esta elección con la métrica de evaluación final, que es la que finalmente dice qué problema de la realidad estamos resolviendo.

Model Selection

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Model Selection y Métrica de evaluación

Model Selection (o Model Validation según la referencia) es cuando realizamos un entrenamiento robusto. Si encontramos la mejor Regresión Logística (por poner un ejemplo) para un único conjunto de datos de entrenamiento, corremos el riesgo de que ese modelo esté *muy atado* a ese conjunto particular de datos y a ese único entrenamiento que hicimos.

Para poder decidir cómo hacer comparaciones robustas y poder elegir un modelo final, necesitamos elegir una métrica de evaluación. Si bien es un paso previo a realizar Model Selection, es tan importante que vamos a dedicar un tiempo a entender lo que implican las diferentes opciones.

Model Selection

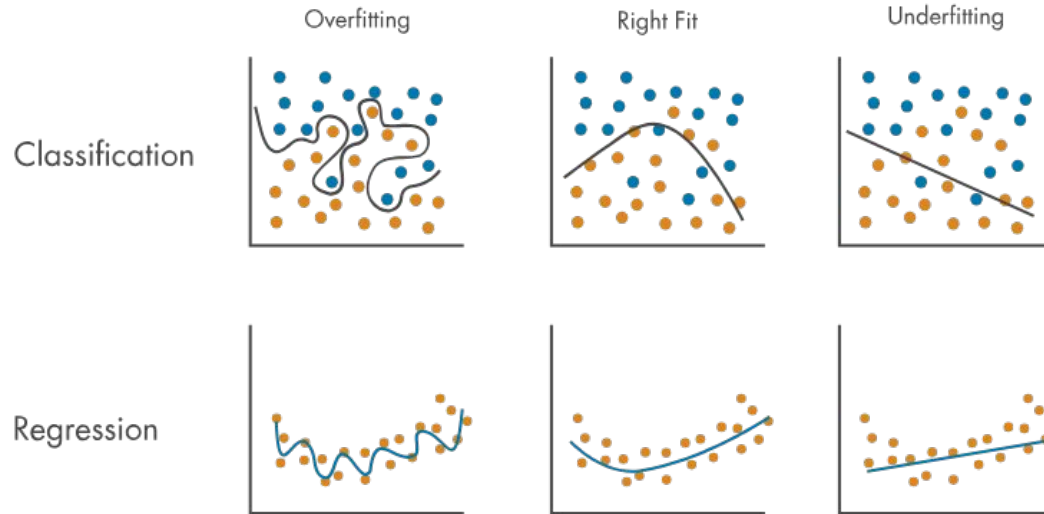
Supongamos que tenemos definida una métrica para medir la calidad de nuestro modelo (ej: Accuracy para clasificación).

¿Qué hago?

Una opción sería agarrar todos los datos disponibles y buscar (supongamos que es posible) los mejores parámetros e hiperparámetros del modelo elegido para esos datos.

Si hacemos esto, ¿Qué esperamos que pase cuando nos lleguen datos nuevos del mundo real?

Underfitting y Overfitting

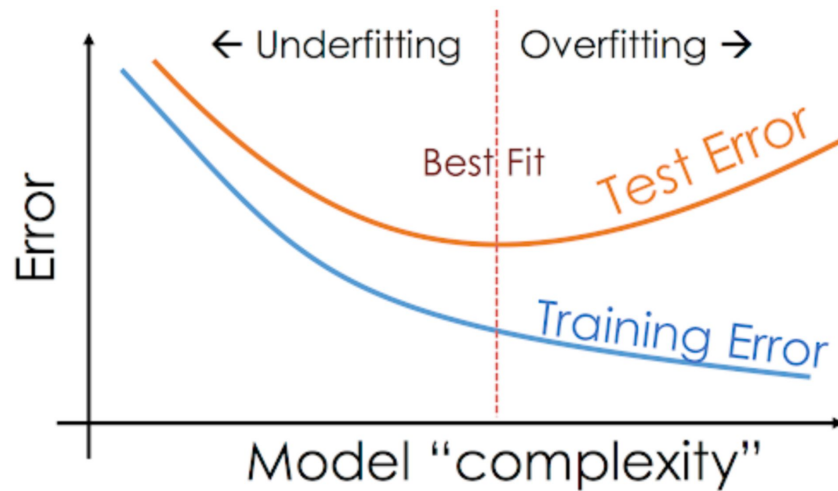


Underfitting: El modelo es tan *rígido* que ni siquiera llega a capturar bien los patrones en el conjunto de datos en el que entrena. Anda mal tanto en el entrenamiento como al generalizar.

Overfitting: El modelo es tan *flexible* que se sobreajusta al conjunto de entrenamiento. Anda bien para esos datos, pero generaliza mal.

Model Selection

Lo que vamos a buscar entonces es que todas las cuestiones a fijar (qué preprocesamiento hacer, qué features usar, qué modelo elegir, cuáles son los mejores parámetros, cuáles son los mejores hiperparámetros, etc) vayan la dirección de minimizar el error al generalizar.



Model Validation

Lo que nos gustaría en un ideal, es saber cómo se va a comportar nuestro modelo al inferir sobre instancias desconocidas, instancias que no utilizamos para definir el modelo. El problema es que justamente no disponemos de ellas (incluso puede haber una limitación temporal).

Vamos entonces a utilizar alguna estrategia en donde simulemos una división entre datos para los que conocemos su variable target y datos para los que no.

A los que sí conozcamos su variable target, los usaremos para entrenar nuestro modelo.

A los que “no conozcamos” su variable target, los usaremos para validar nuestro modelo.

Model Validation

Los datos que tengamos disponibles los vamos entonces a dividir, con diferentes estrategias, en los siguientes conjuntos:

- Training Set: El conjunto de datos que utilizamos para entrenar.
- Validation Set: El conjunto de datos que utilizamos para validar las decisiones que tomamos durante el entrenamiento.
- Test Set: Un conjunto de datos final donde realizaremos la evaluación final. Lo que pase sobre este conjunto no debería influenciar decisiones del entrenamiento (Data Leakage).

Simple Holdout Set

Una primera estrategia es simplemente partir nuestros datos en dos conjuntos:

- Train: valores *grandes* como 80% de los datos.
- Test: datos restantes (aprox 20%).

Es una partición demasiado simple en donde ni siquiera validamos. Entrenamos de la mejor manera posible y finalmente vemos cómo da en el conjunto de test.

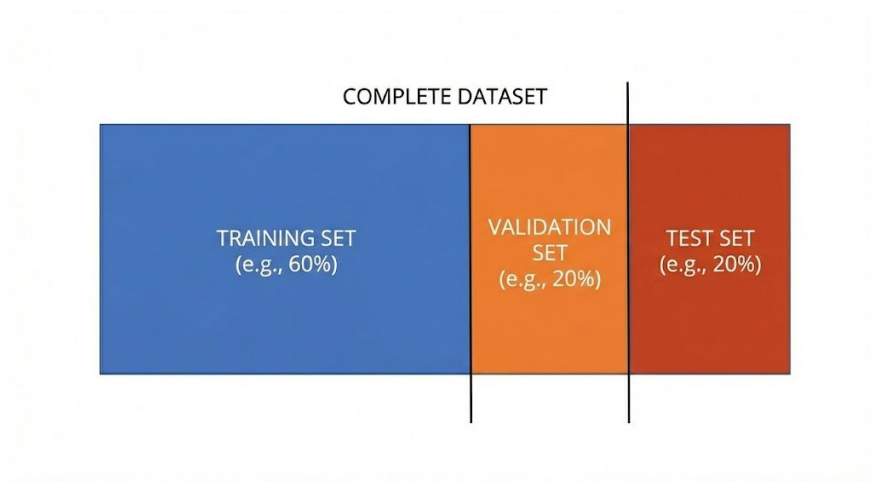
Recomendable exclusivamente para prototipos y problemas pequeños.



Validation Holdout Set

Partimos nuestros datos en 3 conjuntos:

- Train: 60%-80% de los datos los utilizamos para entrenar.
- Validation: 10%-20% de los datos los utilizamos para validar.
- Test: 10%-20% de los datos los utilizamos para la evaluación final.



Validation Holdout Set

Si bien ya es una estrategia más robusta tiene algunos problemas:

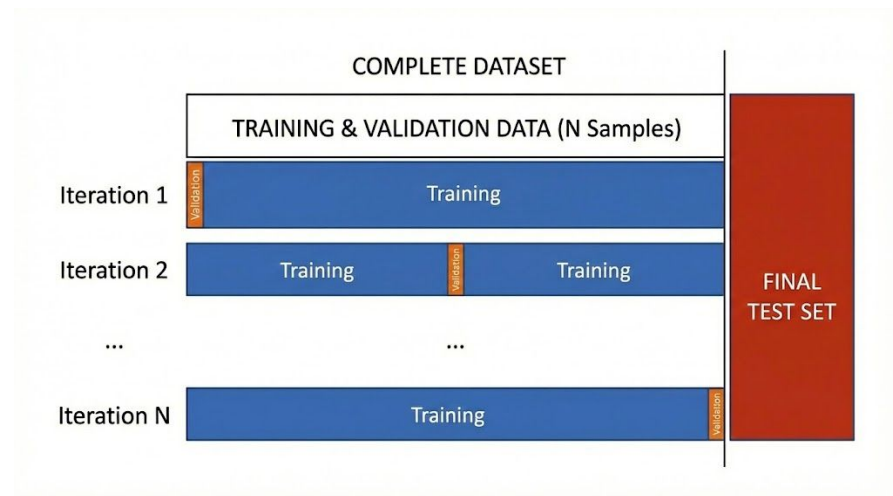
- Si el conjunto de datos es chico, esta partición nos puede dejar pocos datos para entrenar.
- Estamos librados a la suerte de una partición en particular en donde las muestras pueden caer de manera desbalanceada en train o validation. Una opción para mitigar esto es repetir el proceso más de una vez, se denomina Repeated Validation Set.

Varios autores argumentan que si la cantidad de datos es suficiente, esta estrategia ya es correcta.

Leave-one-out Cross-Validation

Una opción para no achicar el conjunto de entrenamiento, es simplemente dejar una sola muestra afuera para validar.

Esto lo hacemos para todas las muestras y luego promediamos la performance sobre todas las opciones.



Leave-one-out Cross-Validation

Ventajas:

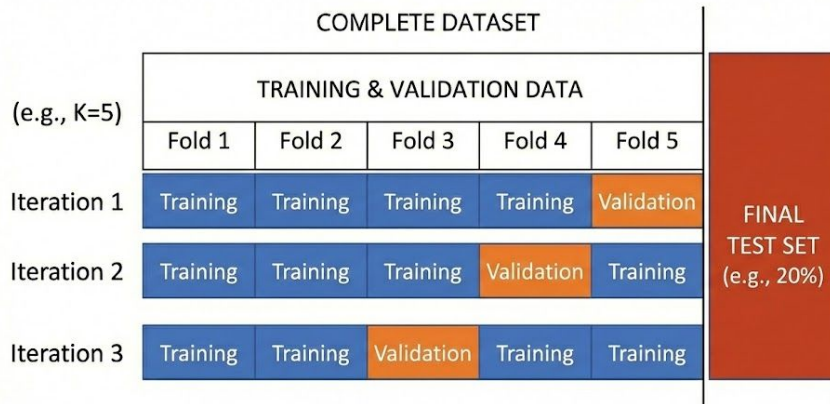
- No dependemos del azar de la partición.
- Podemos usar más muestras para entrenar.
- Cada observación se usa la misma cantidad de veces para validar (1).

Desventajas:

- Más allá de que sea paralelizable, requiere un tiempo de cómputo altísimo. Solo es viable para datasets muy chicos.
- Hay propiedades estadísticas de este método que lo hacen menos adecuado que otras alternativas.

K-fold Cross-Validation

Es la versión menos extrema de LOOCV. En vez de dejar solo una muestra afuera, vamos a partir el conjunto de datos en k particiones. Usaremos $k-1$ para entrenar y 1 para validar, y luego iremos rotando el uso.



Otras estrategias

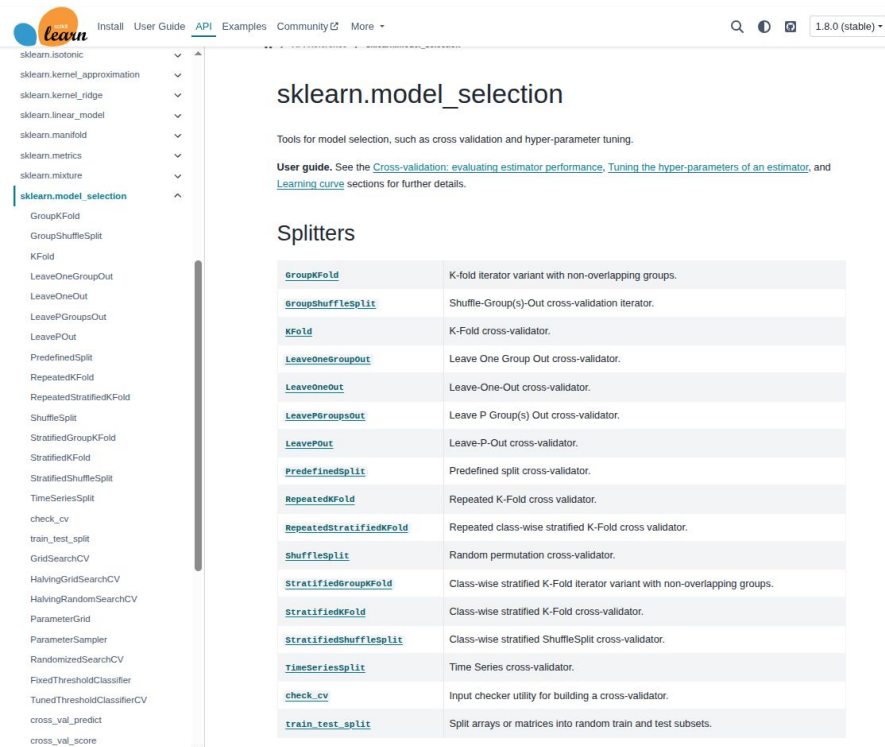
En situaciones particulares, las estrategias presentadas pueden no ser adecuadas e incluso traer problemas (en general, Data Leakages). Existen otras estrategias de validación según los diferentes casos de uso, algunos ejemplos:

- Stratified K-fold: Cuando las clases están desbalanceadas en un problema de clasificación, suele ser deseable mantener el mismo desbalance al generar la partición.
- Time Series Split: Cuando los datos tienen temporalidad, K-fold como fue presentado no tiene sentido. Existen otras estrategias como *rolling window*.
- Leave One Group Out: Agrupar las muestras por alguna identidad (por ejemplo, misma empresa, mismo cliente, mismo paciente) y partir las muestras según esta identidad para asegurarme de que al validar estemos viendo muestras *completamente nuevas*.

Model Selection

Scikit-learn tiene varias funcionalidades para ayudarnos en este proceso.

Se pueden encontrar en https://scikit-learn.org/stable/api/sklearn.model_selection.html



The screenshot shows the scikit-learn website's API documentation for the `sklearn.model_selection` module. The left sidebar lists various modules, with `sklearn.model_selection` selected. The main content area is titled `sklearn.model_selection` and includes a brief description: "Tools for model selection, such as cross validation and hyper-parameter tuning." It also provides a "User guide" link. Below this, the "Splitters" section is highlighted, containing a table of cross-validation splitters.

Splitter	Description
<code>GroupKFold</code>	K-fold iterator variant with non-overlapping groups.
<code>GroupShuffleSplit</code>	Shuffle-Group(s)-Out cross-validation iterator.
<code>KFold</code>	K-Fold cross-validator.
<code>LeaveOneGroupOut</code>	Leave One Group Out cross-validator.
<code>LeaveOneOut</code>	Leave-One-Out cross-validator.
<code>LeavePGroupsOut</code>	Leave P Group(s) Out cross-validator.
<code>LeavePOut</code>	Leave-P-Out cross-validator.
<code>PredefinedSplit</code>	Predefined split cross-validator.
<code>RepeatedKFold</code>	Repeated K-Fold cross validator.
<code>RepeatedStratifiedKFold</code>	Repeated class-wise stratified K-Fold cross validator.
<code>ShuffleSplit</code>	Random permutation cross-validator.
<code>StratifiedGroupKFold</code>	Class-wise stratified K-Fold iterator variant with non-overlapping groups.
<code>StratifiedKFold</code>	Class-wise stratified K-Fold cross-validator.
<code>StratifiedShuffleSplit</code>	Class-wise stratified ShuffleSplit cross-validator.
<code>TimeSeriesSplit</code>	Time Series cross-validator.
<code>check_cv</code>	Input checker utility for building a cross-validator.
<code>train_test_split</code>	Split arrays or matrices into random train and test subsets.

Métricas de evaluación

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Métricas de evaluación del modelo

Las métricas con las que evaluamos nuestros modelos esconden varias decisiones que pueden ser determinantes para el problema a resolver.

Más allá de que algunas métricas tienen algunas propiedades matemáticas a conocer, todas tienen asociada una decisión de *negocio* y son mucho más *subjetivas* de lo que parecen. Son estas métricas las que dicen qué problema de la realidad estamos resolviendo.

Supongamos que estamos haciendo un regresor para predecir precios inmobiliarios. Si el regresor predice perfectamente todos los precios, cualquier métrica razonable me va a dar lo mismo, ¿y cuando no?

Dudas preliminares

- En regresión hay métricas que penalizan el error lineal entre la predicción y la realidad, y hay métricas que penalizan el error logarítmico. ¿Por qué hay diferentes? ¿Para qué sirven? ¿Qué es esa *subjetividad* que decimos que esconden?
- En un clasificador, llamemos *accuracy* a la cantidad de muestras bien clasificadas sobre el total. ¿Qué tan útil es pasar de un clasificador de 0.87 a uno de 0.89 de accuracy?
- Si tengo un clasificador que tiene 0.98 de accuracy y otro que tiene 0.99, ¿qué tanto mejor es el segundo?
- ¿Cuál es un ejemplo de clasificación binaria real donde un alto accuracy no indica nada útil?

Métricas de evaluación del modelo

Hay muchas métricas porque hay muchas cosas a medir!

Hay diferentes situaciones reales a ponder, diferentes problemas a resolver o priorizar. En cada una de esas situaciones, la métrica podría ser eventualmente distinta.

Algunas métricas importantes:

- Regresión: MSE, MAE, RMSE, RMSLE, MAPE
 - MSE
 - RMSE
 - RMSLE
 - MAE
 - MAPE
 - R^2
- Clasificación:
 - Accuracy
 - Matriz de Confusión*
 - Precision
 - Recall
 - F-scores
 - ROC y AUC-ROC
 - PR y AUC-PR

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- ¿Qué es? La media de los errores de predicción al cuadrado.
- ¿Qué prioriza? Penaliza más los errores más grandes.
- ¿Cuándo usarla? Cuando equivocarse por mucho es inaceptable.
- ¿Cuándo no usarla? Cuando hay valores atípicos que no queremos ponderar de más.
- Particularidad: Al ser derivable, es el estándar de entrenamiento de muchos métodos.
- Dato suelto: Perdés la unidad y la noción del valor (está al cuadrado, no es grave).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- ¿Qué es? Simplemente la raíz cuadrada del MSE
- ¿Qué prioriza? Idem MSE
- ¿Cuándo usarla? Idem MSE
- ¿Cuándo no usarla? Idem MSE
- Particularidad: Se suele comparar con el MAE para identificar puntos con mucho error.
- Dato suelto: Se recupera la unidad original de comparación.

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

- ¿Qué es? Aplicar logaritmo antes de calcular el RMSE
- ¿Qué prioriza? Resta de logaritmos es logaritmo de la división, pasa a mirar cuestiones relativas en vez de absolutas.
- ¿Cuándo usarla? Cuando me importa el error porcentual de las muestras y no el absoluto.
- ¿Cuándo no usarla? Si los datos son negativos no puede usarlo “de una”.
- Particularidad: Por cómo es la curva del logaritmo, penaliza más por defecto que por exceso.
- Dato suelto: El +1 es simplemente para poder usar el 0 en y .

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- ¿Qué es? La media de las diferencias, mirando el valor absoluto.
- ¿Qué prioriza? Las medianas de las distribuciones, sin darle mucho peso a outliers.
- ¿Cuándo usarla? Cuando querés que la métrica sea directamente interpretable.
- ¿Cuándo no usarla? Cuando no nos interesa el error absoluto o tenemos muestras en magnitudes muy diferentes.
- Particularidad: “Hacia afuera” del área suele ser bien vista aunque “hacia adentro” no.
- Dato suelto: Si no son datos entre 0 y 1, da siempre más chico que el RMSE y la distancia entre las métricas se usa para encontrar errores puntuales.

MAPE

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- ¿Qué es? La media de los errores porcentuales.
- ¿Qué prioriza? Todas las muestras por igual en su error relativo.
- ¿Cuándo usarla? Cuando nos importa el error relativo y nos sirve reportar porcentajes.
- ¿Cuándo no usarla? Si los datos son muy pequeños un error absoluto pequeño puede dar un relativo desproporcionado.
- Particularidad: Los porcentajes no son iguales por exceso que por defecto, *sMAPE corrige* eso.
- Dato suelto: Hay nichos donde es la métrica default (ej: forecasting de ventas)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- ¿Qué es?
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- ¿Qué es? Una medida de que tan bien anda mi modelo con respecto a simplemente predecir la media para todos.
- ¿Qué prioriza? Al ser errores al cuadrado, intenta que no haya errores muy grandes en valor absoluto.
- ¿Cuándo usarla? Cuando queremos comparar modelos en que los datos están en diferente escala.
- ¿Cuándo no usarla? Cuando queremos comparar modelos con diferentes predictores, por ejemplo, dos regresiones lineales con distinta cantidad de features (existe el adjusted R²)
- Particularidad: Un porcentaje altísimo de gente trabajando en DS/ML cree que esta métrica va entre 0 y 1.
- Dato suelto: 1 es una predicción perfecta. 0 es una predicción tan vaga como el promedio.

Accuracy

- ¿Qué es?: Casos bien clasificados sobre cantidad totales de casos.
- ¿Qué prioriza? La tasa general de éxito sin importar de qué clase hablamos.
- ¿Cuándo usarla? Cuando las clases están balanceadas.
- ¿Cuándo no usarla? Cuando ciertos errores nos importan más que otros.
- Particularidad: Una de las métricas más fáciles de comunicar, pero también de las más fáciles de manipular.
- Dato suelto: A veces se confunde con *hit rate* pero en la mayoría de las referencias no son lo mismo.

Matriz de confusión

En problemas de clasificación binaria, es una matriz de 4 elementos que condensa la información de las inferencias realizadas cruzando el valor inferido vs el valor real.

Es importante distinguir cuál es la clase positiva y cuál la negativa.

Se extiende trivialmente a clasificaciones con varias categorías.

Dadas estas definiciones, *accuracy* se puede definir como $(TP+TN)/(TP+FN+FP+TN)$

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Precision

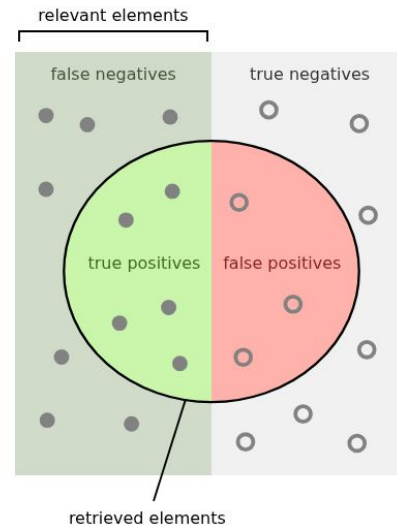
$$Precision = \frac{VP}{VP + FP}$$

- ¿Qué es?:
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

Precision

$$Precision = \frac{VP}{VP + FP}$$

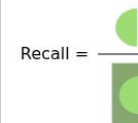
- ¿Qué es?: “¿De los que dije que sí eran, cuántos realmente eran?”
- ¿Qué prioriza? La confianza en decir que sí. Con *precision* alta, cuando el modelo dice sí, seguramente es sí.
- ¿Cuándo usarla? Cuando es muy grave tener falsos positivos.
- ¿Cuándo no usarla? Cuando es muy grave tener falsos negativos.
- Particularidad: ~~“el que no arriesga no gana”~~. En general si no arriesgamos, la *precision* es alta.
- Dato suelto: No se suele mirar sola, cómo mínimo se mira el *recall* también.



How many retrieved items are relevant?



How many relevant items are retrieved?



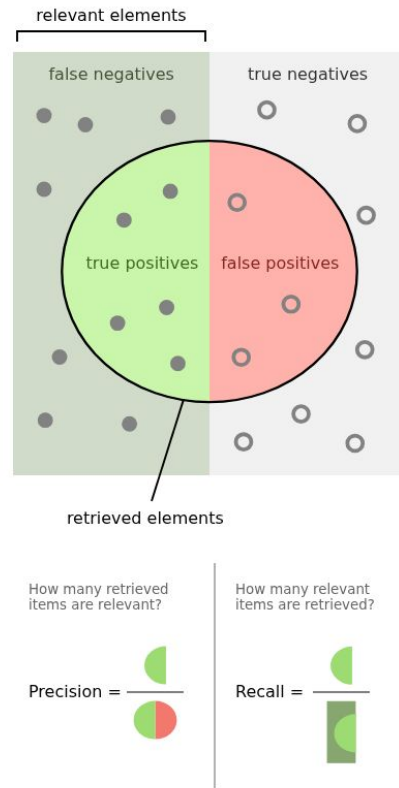
$$Recall = \frac{VP}{VP + FN}$$

- ¿Qué es?:
- ¿Qué prioriza?
- ¿Cuándo usarla?
- ¿Cuándo no usarla?
- Particularidad:
- Dato suelto:

Recall

$$Recall = \frac{VP}{VP + FN}$$

- ¿Qué es?: “De los que sí eran, ¿cuántos dije que sí eran?”
- ¿Qué prioriza? La cobertura de la clase positiva. Por ahí reviso de más, pero que no se me pase nadie.
- ¿Cuándo usarla? Cuando es muy grave tener falsos negativos.
- ¿Cuándo no usarla? Cuando es muy grave tener falsos positivos.
- Particularidad: *return true* va a tener siempre 1.0 de *Recall*.
- Dato suelto: En algunas métricas se lo conoce también como True Positive Rate.



F-scores

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- ¿Qué es?: La media armónica entre *precision* y *recall*.
- ¿Qué prioriza? El balance entre ambas métricas de base.
- ¿Cuándo usarla? ~~Nunca~~ Cuando ambos tipos de errores importan en niveles parecidos.
- ¿Cuándo no usarla? Cuando un tipo de error es más determinante que el otro.
- Particularidad: Nunca mira los *True Negative* (más que por descarte). Hay métricas como el MCC que a veces son preferidas.
- Dato suelto: Existen otros *f-scores* (F2, F3, etc) con una fórmula general para mezclar las métricas.

ROC y AUC-ROC

En general, los modelos para clasificación binaria lo que hacen es arrojar por cada muestra la probabilidad de que pertenezcan a la clase positiva. Luego se utiliza un *threshold* para decidir si la muestra es positiva o no.

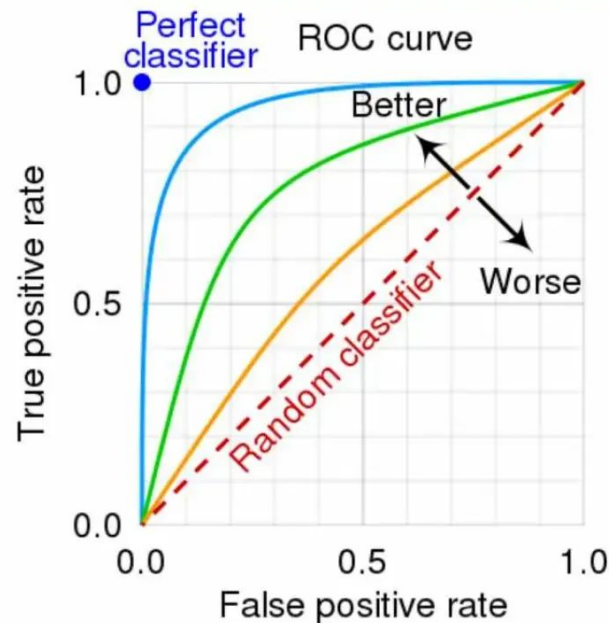
Todas las métricas vistas hasta ahora son muy sensibles al threshold.

Hay métricas que muestran cómo se comporta el modelo a medida que vamos variando el threshold.

Una de las más conocidas es la curva ROC (Receiving Operating Characteristic).

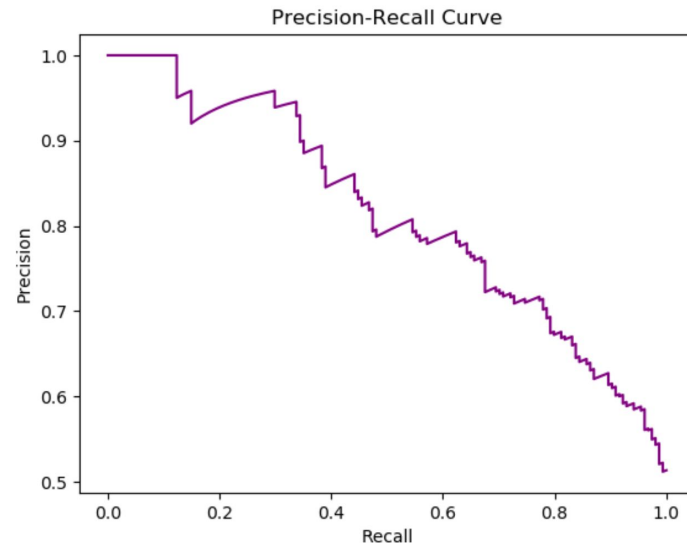
ROC y AUC-ROC

- ¿Qué es?: TPR (TP / P , o sea, Recall) vs FPR (FP / N)
- ¿Qué prioriza? La capacidad de que las muestras queden *bien ordenadas* más allá de dónde está el corte.
- ¿Cuándo usarla? Para comparar modelos de clasificación binaria con una sola métrica.
- ¿Cuándo no usarla? Cuando las clases están muy desbalanceadas, es fácil conseguir una buena curva sin un buen predictor.
- Particularidad: Más que la curva, muchas veces se usa el *área bajo la curva* (AUC), para ver qué tan lejos estamos de un predictor random.
- Dato suelto: Se inventó en un contexto de uso muy particulares (radares) que le da el nombre.



PR y AUC-PR

- ¿Qué es?: Graficar *precision vs recall* variando el threshold.
- ¿Qué prioriza? Las métricas sobre la clase positiva.
- ¿Cuándo usarla? Cuando el dataset está muy desbalanceado.
- ¿Cuándo no usarla? Cuando los True Negative importan tanto como los positivos.
- Particularidad: No hay una curva fija de un predictor random, depende del balance de las clases.



No todo es Regresión y Clasificación en la vida (o en realidad sí, pero después lo charlamos).

Hay métricas específicas para otros tipos de problemas, algunos ejemplos:

- Object Detection: IOU, mAP.
- Ordering: Mean Reciprocal Rank.
- Traducción: BLEU.

Diferencia entre métricas

Al conocer ahora casi todo el camino, vemos que hay dos métricas. Por un lado la métrica de optimización que el modelo usa para encontrar los mejores parámetros. Por otro lado la métrica de evaluación que utilizamos para ver la performance del modelo.

¿Tiene sentido que haya dos? ¿No deberían ser la misma para “ir en la misma dirección”? ¿Son dos distintas o es una sola? ¿Son la misma? ¿Pueden ser la misma?

Puesta en producción

- Definición del problema.
- Adquisición de datos.
- Preprocesamiento de los datos.
- Selección de la clase de modelo.
- Algoritmo de optimización del modelo.
- Métrica de optimización del modelo.
- Model Selection y Optimización de hiperparámetros.
- Métricas de evaluación del modelo.
- Puesta en producción.

Puesta en producción

Tenemos nuestro modelo hermoso súper generalizador del universo, ¿y después qué?

Existe todo un mundo relacionado al desarrollo de software e infraestructura para disponibilizar estos modelos (MLOps, Machine Learning Engineering).

Algunas preguntas de esta sección pueden condicionar a pasos anteriores del camino.

¿Voy a tener todos los datos necesarios al momento de usar el modelo?

¿Qué tiempo de inferencia es razonable para mi aplicación?

Además, el uso prolongado en el tiempo de modelos de Machine Learning implica desafíos técnicos-matemáticos a resolver (data shifts).

O'REILLY®

Designing Machine Learning Systems

An Iterative Process
for Production-Ready
Applications



Chip Huyen